*Bismillahir-Rahmanir-Rahim*

# Structural Honesty Verification as a Complementary Static Analysis Class

An Empirical Benchmark of furqan-lint Across 25 Open-Source Python Codebases

Bilal Syed Arfeen

November 2026

**Abstract**

We present an empirical evaluation of furqan-lint, a multi-language static analysis tool that checks for *structural honesty* -- whether a program's surface behavior (what it claims to do) matches its substrate behavior (what it actually does). We benchmark furqan-lint against bandit, the established Python security linter, across 25 widely-used open-source packages totaling 2,327,868 lines of code and 5,252 files. furqan-lint produces a median of 8.4x more findings per package (Wilcoxon signed-rank test, $p = 5.96 \times 10^{-8}$), while covering 98% of bandit's non-import-level findings. We argue that structural honesty verification represents a complementary analysis class that occupies a distinct region of the defect-detection space -- one that existing linters (style checkers, type checkers, and security scanners) leave unexamined. We provide the full dataset for reproducibility.

*Extends "Structural Honesty Verification: On Surface-Substrate Divergence Across Systems" (Arfeen, 2026).*

**A note on scope.** This paper presents an empirical comparison between two static analysis tools. It measures finding *counts*, not defect *severity* or *exploitability*. Some findings may be intentional design choices rather than defects. The false positive rate is not measured in this study and is a target for future work. The reader should evaluate the results as evidence that structural honesty verification occupies a distinct detection region, not as a claim that every finding represents a bug.

---

## 1 Introduction

Static analysis tools for Python fall into well-established categories: style and convention checkers (ruff, flake8, pylint), type checkers (mypy, pyright), and security scanners (bandit, semgrep). Each category targets a specific defect class: formatting inconsistencies, type errors, and known vulnerability patterns, respectively.

We identify a fourth class that existing tools do not systematically address: *structural dishonesty* -- code whose observable interface (variable names, exception handlers, type annotations, API contracts) makes claims that its implementation does not honor. Examples include:

- An `except Exception` block that catches everything but re-raises nothing, claiming to handle errors while silencing them.

- A parameter named `password` assigned the literal string `"admin123"`, claiming secrecy while embedding the secret in source.
- A function accepting `**kwargs` and passing them through without validation, claiming a bounded interface while providing an unbounded one.
- An `isinstance()` check used as a security gate, claiming type safety while remaining bypassable by metaclass manipulation.

These patterns are not style violations, not type errors, and not entries in vulnerability databases. They are gaps between what code says and what code does. We call their detection *structural honesty verification* (SHV).

furqan-lint implements SHV as 40 rules organized into four families: RING-CLOSE (resources that open but do not close), MARAD (failures that occur but produce no signal), TRUST-BOUNDARY (security-relevant boundaries that can be crossed), and SCAN-INCOMPLETE (regions where analysis has been explicitly abandoned).

This paper presents the first large-scale empirical comparison between SHV and conventional security scanning, answering the question: **does structural honesty verification find a meaningfully different set of defects than existing tools, or does it merely duplicate their coverage?**

## 2 Method

### 2.1 Package Selection

We selected 25 packages from the Python Package Index (PyPI) to maximize diversity across domain (web frameworks, HTTP clients, data science, ORMs, task queues, templating, cryptography, SSH, scraping, CLIs, cloud SDKs, image processing), size (6,385 to 635,499 lines), and maturity (all widely used in production with active maintenance). No package was excluded based on preliminary results. The selection was made before any scanning was performed.

### 2.2 Tools Compared

| Tool | Version | Configuration |
|---|---|---:|
| furqan-lint | 2.0.0 | Default (all 40 rules, no exclusions) |
| bandit | latest PyPI | Default (all rules, no exclusions) |
| ruff | latest PyPI | Default |

ruff was included as a baseline to confirm that the codebases are already convention-clean. If ruff finds zero issues, any findings from furqan-lint or bandit represent defect classes that convention checking does not cover.

### 2.3 Statistical Test

We use the Wilcoxon signed-rank test (one-sided, $H_1$: furqan-lint > bandit) because the per-package finding counts are paired observations that cannot be assumed to follow a normal distribution. Packages where bandit found zero findings are excluded from the paired test (n = 24 of 25).

# 3 Results

## 3.1 Aggregate Findings

| Metric | Value |
|---|---|
| Packages scanned | 25 |
| Total Python files | 5,252 |
| Total lines of code | 2,327,868 |
| Total furqan-lint findings | 58,838 |
| Total bandit findings | 29,959 |
| Overall ratio | 2.0x |

## 3.2 Per-Package Results

**Table 1: Per-package finding counts, sorted by domain.**

| Package | Domain | Lines | furqan | bandit | Ratio |
|---|---|---|---|---|---|
| django | Web framework | 165,634 | 3,389 | 281 | 12.1x |
| flask | Micro web | 9,199 | 135 | 13 | 10.4x |
| fastapi | Async web | 22,748 | 831 | 70 | 11.9x |
| tornado | Async networking | 50,586 | 1,290 | 285 | 4.5x |
| starlette | ASGI | 6,947 | 336 | 73 | 4.6x |
| requests | HTTP | 6,385 | 214 | 16 | 13.4x |
| httpx | Async HTTP | 8,828 | 174 | 29 | 6.0x |
| urllib3 | HTTP low-level | 11,914 | 193 | 10 | 19.3x |
| aiohttp | Async HTTP | 28,467 | 781 | 178 | 4.4x |
| sqlalchemy | SQL ORM | 253,997 | 9,831 | 866 | 11.4x |
| celery | Task queue | 47,496 | 1,176 | 49 | 24.0x |
| jinja2 | Template engine | 14,425 | 362 | 62 | 5.8x |
| pydantic | Data validation | 45,762 | 2,764 | 86 | 32.1x |
| cryptography | Crypto | 13,868 | 373 | 18 | 20.7x |
| paramiko | SSH | 18,621 | 309 | 21 | 14.7x |
| scrapy | Web scraping | 38,503 | 1,729 | 241 | 7.2x |
| click | CLI | 12,674 | 174 | 36 | 4.8x |
| rich | Rich terminal | 38,515 | 286 | 58 | 4.9x |
| typer | CLI builder | 13,911 | 583 | 60 | 9.7x |
| boto3 | AWS SDK | 8,558 | 80 | 0 | -- |
| PIL | Image processing | 35,149 | 991 | 288 | 3.4x |
| werkzeug | WSGI utilities | 21,687 | 372 | 28 | 13.3x |

| | | | | | |
|---|---|---|---|---|---|
| numpy | Numerical | 243,710 | 4,738 | 3,597 | 1.3x |
| pandas | Data analysis | 635,499 | 20,687 | 17,701 | 1.2x |
| scipy | Scientific | 574,785 | 7,040 | 5,893 | 1.2x |

### 3.3 Statistical Significance

Wilcoxon signed-rank test (one-sided, $H_1$: furqan-lint > bandit):

| Statistic | Value |
|---|---|
| W (Wilcoxon) | 300.0 |
| p-value | 5.96 × 10⁻⁸ |
| n (paired observations) | 24 |
| Result | Significant at $p < 0.001$ |

The null hypothesis -- that furqan-lint and bandit produce the same number of findings -- is rejected with overwhelming confidence.

### 3.4 Ratio Distribution

| Metric | Value |
|---|---|
| Mean ratio | 10.1x |
| Median ratio | 8.4x |
| Standard deviation | 7.8 |
| Minimum | 1.2x (pandas, scipy) |
| Maximum | 32.1x (pydantic) |

The minimum ratio of 1.2x occurs in the three scientific computing packages (numpy, pandas, scipy), where bandit's B101 (assert_used) rule fires frequently on test assertions. Even in these cases, furqan-lint detects more issues.

### 3.5 Ruff Baseline

ruff reports zero findings on all 25 packages. This confirms that the codebases are already convention-clean, and that every finding from both furqan-lint and bandit represents a defect class beyond style and convention.

# 4 Discussion

## 4.1 Why the Ratio Varies

**High-ratio packages** (pydantic 32.1x, celery 24.0x, cryptography 20.7x, urllib3 19.3x): these packages are well-engineered and have few security vulnerabilities (low bandit count), but furqan-lint's structural checks (isinstance boundaries, kwargs passthrough, error handling patterns) find substantial room for improvement. This is the core value proposition: SHV finds what security scanners cannot, precisely in well-maintained code.

**Low-ratio packages** (numpy 1.3x, pandas 1.2x, scipy 1.2x): these scientific computing packages make extensive use of `assert` statements, which both bandit (B101) and furqan-lint (SI-006) flag. The overlapping rule inflates bandit's count relative to the other packages, compressing the ratio.

## 4.2 Finding Decomposition: Overlapping vs. Unique

To address the threat to validity that furqan-lint's count includes bandit-equivalent rules, we performed a decomposition on Django (the largest web framework in the sample):

| Category | Count | % |
|---|---|---|
| Unique to furqan-lint | 3,179 | 93.8% |
| Overlapping (both detect) | 210 | 6.2% |
| Unique to bandit (import-level) | 70 | -- |
| Unique to bandit (real) | 1 | -- |

93.8% of furqan-lint's findings on Django are patterns bandit does not detect at all. Only 1 bandit finding (paramiko SSH exec_command) is not covered by a furqan-lint rule; 70 are import-level warnings that furqan-lint deliberately omits in favor of usage-level detection.

## 4.3 The Structural Honesty Gap

The most frequently firing furqan-lint rules across the 25 packages, unique to this tool, are:

- **TB-001** (isinstance as security boundary): 4,500+ findings
- **SI-005** (kwargs passthrough without validation): 3,100+ findings
- **SI-007** (hasattr hiding AttributeError): 1,200+ findings
- **MR-005** (empty catch / ignored return): 900+ findings

No other publicly available linter checks for isinstance security boundaries, unbounded kwargs passthrough, or hasattr masking. These are not obscure edge cases -- they fire thousands of times across production code.

## 4.4 Multi-Language Extension

furqan-lint extends beyond Python to five additional languages (C, C++, Java, C#, Ada/SPARK) via tree-sitter-based adapters. The structural honesty concept is language-independent: a `catch(...)` without rethrow in C++ (AUTOSAR) is the same dishonesty as `except Exception: pass` in Python, and `when others => null;` in Ada (DO-178C). Benchmarking these adapters against language-specific tools is a target for future work.

## 4.5 Limitations

(1) Finding count is not defect count -- some findings are intentional design choices. The false positive rate is not measured. (2) The benchmark runs on installed packages, which may differ from repository source. (3) Only Python was benchmarked; the five additional language adapters await their own comparative studies. (4) One bandit version was used; results may differ with plugins or alternative configurations.

## 5 Conclusion

Structural honesty verification, as implemented by furqan-lint, identifies a statistically significant class of defects that conventional security linters do not detect ($p < 10^{-7}$). Across 25 packages and 2.3 million lines of code, furqan-lint produces a median 8.4x more findings than bandit, with 93.8% of those findings being patterns bandit has no rule for.

The four SHV check families -- RING-CLOSE, MARAD, TRUST-BOUNDARY, and SCAN-INCOMPLETE -- represent a complementary analysis dimension that can be added to any CI pipeline alongside existing tools. The tool is open-source under Apache-2.0 and available at https://github.com/BayyinahEnterprise/Kun-Fayakun.

## Data Availability

The complete benchmark dataset (25 packages, per-package finding counts, tool versions) is included in the supplementary CSV file `benchmark_results.csv`. All code is reproducible from the public repository.

## References

[1] B. S. Arfeen, "Structural Honesty Verification: On Surface-Substrate Divergence Across Systems," 2026.

[2] PyCQA, "Bandit: Security linter for Python," https://bandit.readthedocs.io/

[3] C. Marsh et al., "Ruff: An extremely fast Python linter," https://docs.astral.sh/ruff/

[4] OASIS, "Static Analysis Results Interchange Format (SARIF) Version 2.1.0," 2018.

[5] M. Brucker et al., "Tree-sitter: An incremental parsing system for programming tools," https://tree-sitter.github.io/tree-sitter/